

Inhaltskonzept & Grundlagenrecherche zur Bachelorarbeit

LED-Produktkonfigurator – Wissenstransfer, Konzeption und
prototypische Umsetzung (configuLED, LANG AG)

Arbeitsunterlage – ausformulierte Regeln, Formeln und Inhalte je Gliederungspunkt
Stand: 31.07.2026

Inhaltsverzeichnis

- 1 Einleitung
 - 1.1 Problemstellung und Motivation
 - 1.2 Zielsetzung und Forschungsfrage
 - 1.3 Aufbau der Arbeit
- 2 Theoretische Grundlagen
 - 2.1 LED-Technik im Veranstaltungsbereich
 - 2.2 Produktkonfiguratoren als interaktive Medienanwendungen

2.3 Wissenstransfer – von Expertenwissen zu nutzbaren Interfaces

2.4 KI-gestützte Softwareentwicklung

3 Analyse des Ist-Zustands

3.1–3.5 (Methodik-Gerüst, empirisch zu füllen)

4 Konzeption

4.1 Anforderungsdefinition

4.2 Informationsarchitektur und Nutzerführung

4.3 UX/UI-Konzept

4.4 Visualisierungskonzept

5 Prototypische Umsetzung

5.1 Technologie-Stack

5.2 KI-gestützte Entwicklung

5.3 Umsetzung des Prototyps

6 Evaluation

6.1 Methodik

6.2 Durchführung und Ergebnisse

6.3 Rückspiegelung an die Anforderungen

Hinweis: Kapitel 3 und 6.2 benötigen echte Erhebungsdaten (Interviews, Tests, Konkurrenzrecherche), die in diesem Dokument nicht erfunden, sondern nur methodisch vorstrukturiert werden.

1 Einleitung

1.1 Problemstellung und Motivation

LED-Wände für den Veranstaltungsbereich sind hochgradig konfigurierbare Produkte: Pixelpitch, Modulmaße, Montageart, Indoor-/Outdoor-Eignung, feste vs. temporäre Installation, Kurvbarkeit und Traglastanforderungen (Traversen/Rigging) ergeben zusammen einen kombinatorisch großen Lösungsraum. Dieses Wissen liegt in der Praxis überwiegend implizit bei Produktmanagement und Vertrieb (LANG AG) – Kundenanfragen werden individuell und manuell geprüft.

Daraus ergibt sich ein doppeltes Problem: Erstens ist der Beratungsprozess personalgebunden, zeitintensiv und schlecht skalierbar. Zweitens sind Kunden ohne technisches Fachwissen (z. B. Veranstalter, die selten LED-Wände anmieten) kaum in der Lage, technisch valide Konfigurationen selbst zusammenzustellen – das Risiko fachlich inkorrektur Anfragen (z. B. unzulässige Kombination aus Outdoor-Einsatz und nicht IP-geschütztem Modul, oder Überschreitung zulässiger Traglasten bei Hängung) steigt.

Der bestehende Konfigurator *configuLED* deckt diesen Bedarf nur unvollständig ab (siehe Ist-Analyse, Kapitel 3). Die Motivation der Arbeit ist es, das bei LANG AG vorhandene Expertenwissen systematisch zu erfassen und in ein nutzbares, selbsterklärendes Interface zu überführen – unter Einsatz KI-gestützter Entwicklungswerkzeuge, um die Umsetzung innerhalb des Bachelorarbeits-Zeitrahmens realistisch zu machen.

Ergänze hier unbedingt firmenspezifische, belastbare Fakten (z. B. durchschnittliche Bearbeitungszeit einer Anfrage, Anzahl Supportanfragen/Monat, Fehlerquote) – das macht den Abschnitt aus einer allgemeinen Problemstellung zu einer konkreten, prüfaren Motivation.

1.2 Zielsetzung und Forschungsfrage

Die Arbeit verfolgt ein zweigeteiltes Ziel, wie es für gestaltungsorientierte (Design-Science-)Arbeiten typisch ist:

1. **Gestaltungsziel:** Konzeption und prototypische Umsetzung eines webbasierten LED-Konfigurators, der die Konfigurationslogik von *configuLED* ablöst bzw. erweitert.
2. **Erkenntnisziel:** Verstehen, wie Expertenwissen methodisch in Konfigurationsregeln und Interface-Logik überführt werden kann, und welchen Effekt dies auf Usability und Konfigurationssicherheit hat.

Formulierungsvorschlag für die zentrale Forschungsfrage:

„Wie muss ein webbasierter LED-Produktkonfigurator gestaltet sein, damit Nutzer:innen ohne spezifisches Fachwissen technisch valide Konfigurationen erstellen können, und welchen Beitrag leistet KI-gestützte Softwareentwicklung zur prototypischen Umsetzung eines solchen Systems?“

Empfehlung: daraus 2–3 Unterforschungsfragen ableiten, die jeweils einem Kapitel zugeordnet werden können, z. B.:

- UF1 (Wissenstransfer): Wie lässt sich implizites Konfigurationswissen aus Vertrieb/Produktmanagement in explizite, prüfbare Regeln überführen? → Kapitel 2.3, 4.1
- UF2 (UX/IA): Welche Informationsarchitektur und Nutzerführung reduziert die wahrgenommene Komplexität für fachfremde Nutzer:innen? → Kapitel 4.2, 4.3, 6
- UF3 (KI-Entwicklung): Wie verändert der Einsatz agentischer KI-Coding-Tools den Entwicklungsprozess eines solchen Prototyps? → Kapitel 5.2

Methodischer Rahmen: **Design Science Research** (Hevner et al. 2004; Peffers et al. 2007) – rechtfertigt den iterativen Aufbau Analyse → Konzeption → Artefakt (Prototyp) → Evaluation.

1.3 Aufbau der Arbeit

Kapitel 2 legt die theoretischen Grundlagen zu LED-Technik, Produktkonfiguratoren, Wissenstransfer und KI-gestützter Entwicklung. Kapitel 3 analysiert den Ist-Zustand bei LANG AG, den bestehenden Konfigurator configuLED, den Wettbewerb sowie die Zielgruppe. Kapitel 4 leitet daraus ein Konzept ab (Anforderungen, Informationsarchitektur, UX/UI, Visualisierung). Kapitel 5 beschreibt die prototypische Umsetzung inklusive des Einsatzes KI-gestützter Entwicklungswerkzeuge. Kapitel 6 evaluiert den Prototyp und spiegelt die Ergebnisse gegen die in Kapitel 4 definierten Anforderungen zurück, um die Forschungsfrage aus 1.2 zu beantworten.

Diesen Abschnitt am besten zuletzt final formulieren, wenn die Kapitelstruktur nicht mehr verändert wird.

2 Theoretische Grundlagen

2.1 LED-Technik im Veranstaltungsbereich

Dieser Abschnitt liefert das technische Vokabular, das später in der Datenbank- und Konfigurationslogik (Kapitel 5) direkt wiederauftaucht.

Pixelpitch und Auflösung

Der **Pixelpitch (P)** bezeichnet den Abstand zweier benachbarter LED-Mittelpunkte in Millimetern und ist die zentrale Kenngröße einer LED-Wand.

Panel-Auflösung: $R_x = \text{Breite}_{\text{Panel}} [\text{mm}] / P$ $R_y = \text{Höhe}_{\text{Panel}} [\text{mm}] / P$
 Gesamtauflösung bei n Modulen: $R_{x,\text{gesamt}} = (n_x \cdot \text{Modulbreite} [\text{mm}]) / P$

Mindestbetrachtungsabstand (MVD)

Aus dem Auflösungsvermögen des menschlichen Auges (ca. 1 Bogenminute) lässt sich der physikalisch sinnvolle Mindestabstand herleiten:

$\text{MVD} [\text{mm}] = P [\text{mm}] / \tan(1') \approx P [\text{mm}] \times 3438$
 $\Rightarrow \text{MVD} [\text{m}] \approx P [\text{mm}] \times 3,4$

In der Branche wird meist eine vereinfachte Faustformel verwendet: ca. 1–3 Meter Betrachtungsabstand je Millimeter Pixelpitch (untere Grenze “technisch noch nutzbar”, obere Grenze “komfortabel”). Für die Bachelorarbeit empfiehlt sich, beide Herleitungen – die physikalisch-exakte und die Industrie-Faustformel – gegenüberzustellen und zu diskutieren, warum Hersteller meist die konservativere Faustformel kommunizieren.

Leistungsaufnahme und Gewicht

Leistungsaufnahme gesamt: $P_{\text{gesamt}} = \sum P_{\text{Modul},i}$
 Ø-Last im Betrieb $\approx 30\text{--}50\%$ der maximalen Leistung (abhängig vom Content)
 Gesamtgewicht: $G_{\text{gesamt}} = n \cdot g_{\text{Modul}}$

Das Gesamtgewicht ist die Eingangsgröße für die statische Prüfung der Traversen-/Hängesysteme (Truss-Mechanik), wie sie im Prototyp in `liam_truss.py` berechnet wird – eine gute Stelle, um in 2.1 kurz auf Traversen-Grundlagen (zulässige Punktlast, Spannweite) einzugehen, bevor das in Kapitel 5.3 konkret wird.

Kurvung (Curving) von LED-Wänden

Modulare LED-Wände lassen sich durch mechanische Scharniere zwischen benachbarten Panels konvex oder konkav kurven. Geometrisch lässt sich eine gekurvte Wand aus n Panels der Breite w mit jeweils gleichem Knickwinkel θ zwischen benachbarten Panels als Sehnenvolygon eines Kreisbogens modellieren:

Radius des Bogens: $R = w / (2 \cdot \sin(\theta/2))$

Gesamtöffnungswinkel: $\theta_{\text{gesamt}} = n \cdot \theta$

Näherung Bogenlänge (kleine θ): $s \approx n \cdot w$

Diese Herleitung gehört fachlich in Kapitel 2, während in Kapitel 4.4/5.3 die bewusste Vereinfachung für die Visualisierung (2D-Illusion statt exakter 3D-Geometrie) als Designentscheidung mit Begründung (Rendering-Aufwand vs. wahrgenommener Mehrwert für den Nutzer) dokumentiert werden sollte – nicht als Abstrich, sondern als bewusste Abwägung.

2.2 Produktkonfiguratoren als interaktive Medienanwendungen

Produktkonfiguratoren sind eine klassische Ausprägung von **Mass Customization** (Piller/Möslein): Sie erlauben es, aus einer begrenzten Menge modularer Komponenten kundenindividuelle Varianten zusammenzustellen, ohne dass jede Variante einzeln entwickelt werden muss.

Kombinatorische Komplexität

Bei k unabhängigen Konfigurationsdimensionen (z. B. Pixelpitch, Montageart, Standort, Kurvung) mit jeweils n_i Optionen ergibt sich die Anzahl theoretisch möglicher Varianten als:

$$N_{\text{Varianten}} = \prod_{i=1}^k n_i$$

Diese kombinatorische Explosion begründet, warum ein rein listenbasierter Ansatz (alle Varianten vorab anlegen) nicht skaliert und ein regelbasiertes System notwendig ist.

Formalisierung als Constraint Satisfaction Problem

Ein Konfigurationsproblem lässt sich formal als Tripel (X, D, C) beschreiben: X = Menge der Variablen (z. B. Montageart, Pixelpitch, Standort), D = Domänen (zulässige Werte je Variable), C = Constraints (z. B. „Standort = outdoor $\Rightarrow$ IP-Schutzklasse $\geq$ IP65“).

Diese Formalisierung ist die theoretische Brücke zur tatsächlichen Implementierung: Im Prototyp entsprechen die Constraints den Boolean-Spalten der Artikeltabelle (`is_indoor_capable`, `is_outdoor_capable`, `is_fixed_capable`, `is_temporary_capable`) und der Filterlogik in `server.py`.

Ergänzend einordnen: Konfiguratoren als **Decision-Support-Systeme**/Guided Selling – der Konfigurator führt aktiv durch den Entscheidungsprozess, statt nur ein Formular zur Datenerfassung zu sein.

2.3 Wissenstransfer – von Expertenwissen zu nutzbaren Interfaces

Zentrale theoretische Grundlage ist die Unterscheidung von implizitem (tacit) und explizitem (explicit) Wissen sowie das **SECI-Modell** nach Nonaka/Takeuchi:

- **Socialization:** Wissen wird zwischen Personen ausgetauscht (z. B. Interview mit Produktmanagement).
- **Externalization:** implizites Wissen wird in explizite Form (Regeln, Dokumentation) überführt.
- **Combination:** explizites Wissen wird systematisiert (z. B. in Entscheidungstabellen, Datenbankschema).
- **Internalization:** das System wird genutzt, wodurch neues implizites Nutzerverständnis entsteht.

Praktisch lässt sich Expertenwissen über **Entscheidungstabellen** und **Business Rules** nach dem Schema `WENN <Bedingung> DANN <Aktion>` formalisieren – das ist unmittelbar auf die Constraint-Logik aus 2.2 und die konkrete Filterlogik in Kapitel 5.3 übertragbar. Referenzliteratur: Wissensmanagement nach Probst/Raub/Romhardt sowie Requirements-Elicitation-Literatur zur Befragung von Fachexperten (z. B. Pohl, „Requirements Engineering“).

2.4 KI-gestützte Softwareentwicklung

Abzugrenzen sind klassische Autocomplete-Assistenten (z. B. frühe Versionen von GitHub Copilot) von **agentischen** KI-Coding-Systemen (z. B. Claude Code), die selbstständig Dateien lesen, Änderungen vornehmen, Kommandozeilen-Tools ausführen und iterativ auf Rückmeldung reagieren können. Relevante theoretische Bausteine:

- **Prompt Engineering** als Kommunikationsform zwischen Entwickler:in und Modell.
- **Human-in-the-loop-Review:** jede KI-generierte Änderung bleibt review-pflichtig.

- **Grenzen:** Halluzination (plausibel klingende, aber falsche Aussagen/Code), Notwendigkeit von Testabdeckung und Codeverständnis trotz KI-Unterstützung.

Zu Produktivitätseffekten von KI-Coding-Assistenten existiert Forschung (u. a. von GitHub/Microsoft Research zu Copilot). Bitte konkrete Studien/Zahlen selbst recherchieren und zitieren – ich vermeide hier bewusst, Studienergebnisse oder Prozentzahlen zu nennen, die ich nicht verifizieren kann.

3 Analyse des Ist-Zustands

Dieses Kapitel erfordert reale Erhebungsdaten (Prozessdokumentation, Konkurrenzrecherche, Interviews). Nachfolgend nur das methodische Gerüst.

3.1 Die LANG AG und ihr Produktmanagement-Prozess

- Unternehmensporträt (Größe, Marktsegment, Produktportfolio).
- Ist-Prozess als Flowchart/BPMN darstellen: Anfrage → Beratung → Konfiguration → Angebot → Bestellung.
- Pain-Point-Liste ableiten (Zeitaufwand je Schritt, Fehlerquellen, Medienbrüche).

3.2 configuLED – Analyse des bestehenden Konfigurators

- Heuristische Evaluation anhand Nielsens 10 Usability-Heuristiken als Prüfraster.
- Feature-Matrix: vorhandene vs. fehlende Funktionen im Vergleich zu den in Kapitel 4 definierten Anforderungen.

3.3 Wettbewerbsanalyse: LED-Konfiguratoren am Markt

Vorschlag für Vergleichskriterien einer Konkurrenzmatrix:

Kriterium	Ausprägung
Konfigurationstiefe	Anzahl konfigurierbarer Dimensionen
Visualisierung	keine / 2D / 3D / Live-Preview
Regelprüfung	keine Validierung / einfache Constraints / vollständige Regel-Engine
Preis-/Angebotsausgabe	ja/nein, automatisiert?
Mobile-Fähigkeit	responsive ja/nein

Konkrete Wettbewerbsprodukte und deren tatsächliche Funktionsumfänge müssen recherchiert werden (Herstellerwebsites, Produktdemos) – das kann ich bei Bedarf per Websuche unterstützen, aber nicht aus dem Gedächtnis behaupten.

3.4 Zielgruppenanalyse

Persona-Methodik mit branchentypischen Rollen als Ausgangshypothese, die durch Kapitel 3.5 (Befragung) zu verifizieren ist:

- **Eventtechniker:in** – hohes technisches Fachwissen, benötigt Detailkontrolle.
- **Verleiher/Bühnenbauer:in** – mittleres Fachwissen, Fokus auf schnelle, korrekte Angebote.
- **Architekt:in/Planer:in** – geringes LED-Fachwissen, Fokus auf visuelles Ergebnis und Machbarkeit.
- **Systemintegrator:in** – hohes Fachwissen, benötigt Exportfunktionen/technische Datenblätter.

3.5 Kundenbefragung zur Konfigurationslogik

- Fragebogendesign mit Likert-Skalen (1–5), Stichprobengröße begründen (z. B. anhand Kundenkontaktliste von LANG AG).
- Auswertung: Mittelwert/Median je Item.

Cronbachs Alpha (Reliabilität mehrerer Items zu einem Konstrukt):

$$\alpha = (k / (k-1)) \cdot (1 - \Sigma \sigma_i^2 / \sigma_{\text{total}}^2)$$

4 Konzeption

4.1 Anforderungsdefinition

- Trennung in funktionale und nicht-funktionale Anforderungen.
- Priorisierung nach **MoSCoW**: Must have / Should have / Could have / Won't have.
- Formulierung als User Stories: Als <Rolle> möchte ich <Ziel>, damit <Nutzen>.
- Jede Anforderung erhält eine eindeutige ID (z. B. FA-01) – wird in Kapitel 6.3 für die Traceability-Matrix benötigt.

4.2 Informationsarchitektur und Nutzerführung

- Methoden: Card Sorting zur Kategorisierung der Konfigurationsschritte, Sitemap, User-Flow-Diagramme.
- Prinzip **Progressive Disclosure**: nicht alle Optionen gleichzeitig zeigen, sondern schrittweise entsprechend Nutzerentscheidung freischalten.

Hick'sches Gesetz (Entscheidungszeit in Abhängigkeit von der Optionsanzahl):

$$RT = a + b \cdot \log_2(n + 1)$$

n = Anzahl gleichzeitig angebotener Auswahlmöglichkeiten; a, b = empirische Konstanten

Diese Formel liefert eine quantitative Begründung dafür, Konfigurationsschritte in mehrere, überschaubare Stufen zu gliedern statt eine große Optionsliste auf einmal zu zeigen – direkt anwendbar auf die Reihenfolge Standort → Einsatzart → Produktserie → Modell, wie sie im Prototyp (/api/products -Filterung) bereits angelegt ist.

4.3 UX/UI-Konzept

- Niensens 10 Usability-Heuristiken als Gestaltungsraster.
- Gestaltgesetze (Nähe, Ähnlichkeit, Kontinuität) zur visuellen Gruppierung von Konfigurationsoptionen.
- Wireframing/Design System als methodischer Zwischenschritt vor der Umsetzung.

WCAG-Kontrastverhältnis (falls Barrierefreiheit behandelt wird):

$$\text{Kontrastverhältnis} = (L_1 + 0,05) / (L_2 + 0,05), \text{ mit } L_1 > L_2 \text{ (relative Luminanz)}$$

Mindestanforderung Normaltext: 4,5 : 1

4.4 Visualisierungskonzept

Anknüpfend an 2.1: Abwägung zwischen geometrisch exakter 3D-Darstellung und einer vereinfachten 2D-Illusion der LED-Wand (insbesondere bei der Krümmungs-Darstellung). Begründete Design-Entscheidung für dieses Projekt: eine einfache 2D-Illusion ist ausreichend und performanter, solange keine konkreten Referenzbilder oder Kundenanforderungen eine exaktere Geometrie verlangen – die in 2.1 hergeleiteten Radius-/Winkel-Formeln bleiben dabei die fachliche Grundlage der Transformation, werden aber bewusst vereinfacht in eine 2D-Canvas-/SVG-Darstellung übersetzt, statt eine vollständige 3D-Engine zu implementieren. Dieser Trade-off (Implementierungsaufwand und Rendering-Performance vs. wahrgenommener Mehrwert für Nutzer:innen) sollte explizit als Entscheidung mit Begründung dokumentiert werden.

5 Prototypische Umsetzung

5.1 Technologie-Stack

Der im Rahmen dieser Arbeit entwickelte Prototyp basiert auf folgendem Stack (Ist-Stand des Repositories):

- **Backend:** Python mit Flask, REST-artige JSON-Endpunkte (z. B. `/api/products`, `/api/mounting-types`, `/api/product-series`).
- **Datenhaltung:** SQLite (`konfigurator.db`), Schema und Stammdaten über mehrere serienspezifische SQL-Setup-Skripte gepflegt (u. a. WP-, RX-, MV-, WT-Serie, Aura Flex) – datengetriebene statt hartkodierte Konfigurationslogik.
- **Frontend:** serverseitig gerendertes HTML/CSS mit Vanilla-JavaScript (`templates/index.html`), kein Frontend-Framework.
- **Architekturmuster:** klassische 3-Schichten-Architektur (Präsentation / Anwendungslogik / Datenhaltung) als Referenzmodell zur Einordnung.

Diesen Abschnitt sinnvoll mit einem Architekturdiagramm (Client ↔ Flask-Server ↔ SQLite) ergänzen.

5.2 KI-gestützte Entwicklung

Dieser Abschnitt kann als reflektierte Prozessbeschreibung des tatsächlichen Vorgehens in diesem Projekt geschrieben werden (Einsatz von Claude Code als agentischem KI-Coding-Tool während der Entwicklung):

- Welche Teilaufgaben wurden per Prompt an das KI-Tool delegiert (z. B. Datenbankschema-Erweiterungen, API-Endpunkte, Filterlogik)?
- Wie sah der Review-/Korrekturprozess aus – wurden Vorschläge unverändert übernommen oder angepasst?
- Konkretes, belegbares Beispiel für nötige menschliche Steuerung: Bei der Visualisierung der Kurvungs-Funktion wurde bewusst auf Referenzbilder gewartet, statt eine vom KI-Modell vorgeschlagene, geometrisch „korrekte“, aber ungetestete Lösung zu übernehmen – ein Beleg dafür, dass fachliche Zielsetzung und Nutzerbedürfnis Vorrang vor technischer Eleganz haben müssen.

Das ordnet sich methodisch als Prozess-/Erfahrungsbericht ein, nicht als reine Tool-Beschreibung, und liefert Antwortmaterial für Unterforschungsfrage UF3 aus 1.2.

5.3 Umsetzung des Prototyps

Beschreibung anhand der tatsächlich implementierten Kernfunktionen:

- **Filterlogik** nach Standort (indoor/outdoor) und Nutzungsart (fest/temporär): Jede Bedingung filtert unabhängig auf ihrer eigenen Spalte, sodass ein „Allrounder“-Produkt (`is_indoor_capable = 1` UND `is_outdoor_capable = 1`) automatisch beide Filter erfüllt – das ergibt das gewünschte logische ODER zwischen den Standortoptionen, ohne dass sich Indoor- und Outdoor-Filter gegenseitig ausschließen. Gutes Beispiel für die Umsetzung der CSP-Constraints aus 2.2 in konkrete SQL-Bedingungen.
- **Serien-/Modellauswahl:** eine Produktserie (`configurator_product`) kann mehrere Modelle/Pixelpitches (`article_catalog_mock`) anbieten – 1:n-Beziehung statt starrer 1:1-Zuordnung.
- **Mechanische Berechnung:** Traversen-/Lastmechanik (`liam_truss.py`) als Umsetzung der in 2.1 hergeleiteten Gewichts- und Traglastformeln.
- **Kurvungs-Feature:** Umsetzung der in 4.4 begründeten vereinfachten 2D-Darstellung.

Ein ER-Diagramm des Datenbankschemas eignet sich hier gut als Abbildung – es macht den Wissenstransfer aus 2.3 sichtbar (Expertenregeln → Tabellenspalten und Constraints).

6 Evaluation

6.1 Methodik

- Usability-Test-Design: Think-Aloud-Protokoll, definierte Testaufgaben, Erfolgsquote je Aufgabe, Zeitmessung.

System Usability Scale (SUS) – 10 Items, 5-stufige Likert-Skala (1 = starke Ablehnung, 5 = starke Zustimmung):

ungerade Items (1,3,5,7,9): Beitrag = Antwort – 1

gerade Items (2,4,6,8,10): Beitrag = 5 – Antwort

SUS-Score = (Summe aller Beiträge) $\times$ 2,5 $\rightarrow$ Skala 0–100

Benchmark: Werte > 68 gelten als überdurchschnittlich (Bangor/Kortum/Miller); zusätzlich einordbar über Sauros Grading-Skala (A–F).

6.2 Durchführung und Ergebnisse

Erfordert reale Testdurchführung. Methodisch sinnvoll: deskriptive Statistik (Mittelwert, Standardabweichung, ggf. grafisch als Boxplot), sowie – falls möglich – Vergleich Alt-Konfigurator (configuLED) vs. neuer Prototyp anhand identischer Testaufgaben.

6.3 Rückspiegelung an die Anforderungen

Abschließende **Requirements-Traceability-Matrix**: jede Anforderungs-ID aus 4.1 wird gegen das jeweilige Testergebnis abgeglichen (erfüllt / teilweise erfüllt / nicht erfüllt). Das schließt den Design-Science-Kreis und beantwortet die Forschungsfrage aus 1.2 direkt und nachvollziehbar.

Offene Punkte – nicht erfindbar, sondern zu erheben

- Kapitel 3.1–3.5: reale Prozess-, Wettbewerbs- und Befragungsdaten von LANG AG.
- Kapitel 6.2: tatsächliche Testdurchführung und Messwerte.
- Konkrete Literaturquellen/Zitate zu 2.4 (KI-Produktivitätsforschung) sollten eigenständig geprüft und zitiert werden.